StarData access console (ZITADEL): technical developer guide

Nov 6, 2026 · @John

Purpose and scope

This guide is the starting point for the team that builds StarData sign-in and access management on our on-premises Kubernetes platform. It turns the 12 screens on the StarData access console (ZITADEL) canvas into deployments, configuration, services, APIs and tests. Earlier decisions come from the access-control design and the ZITADEL and RBAC design doc.

What you build:

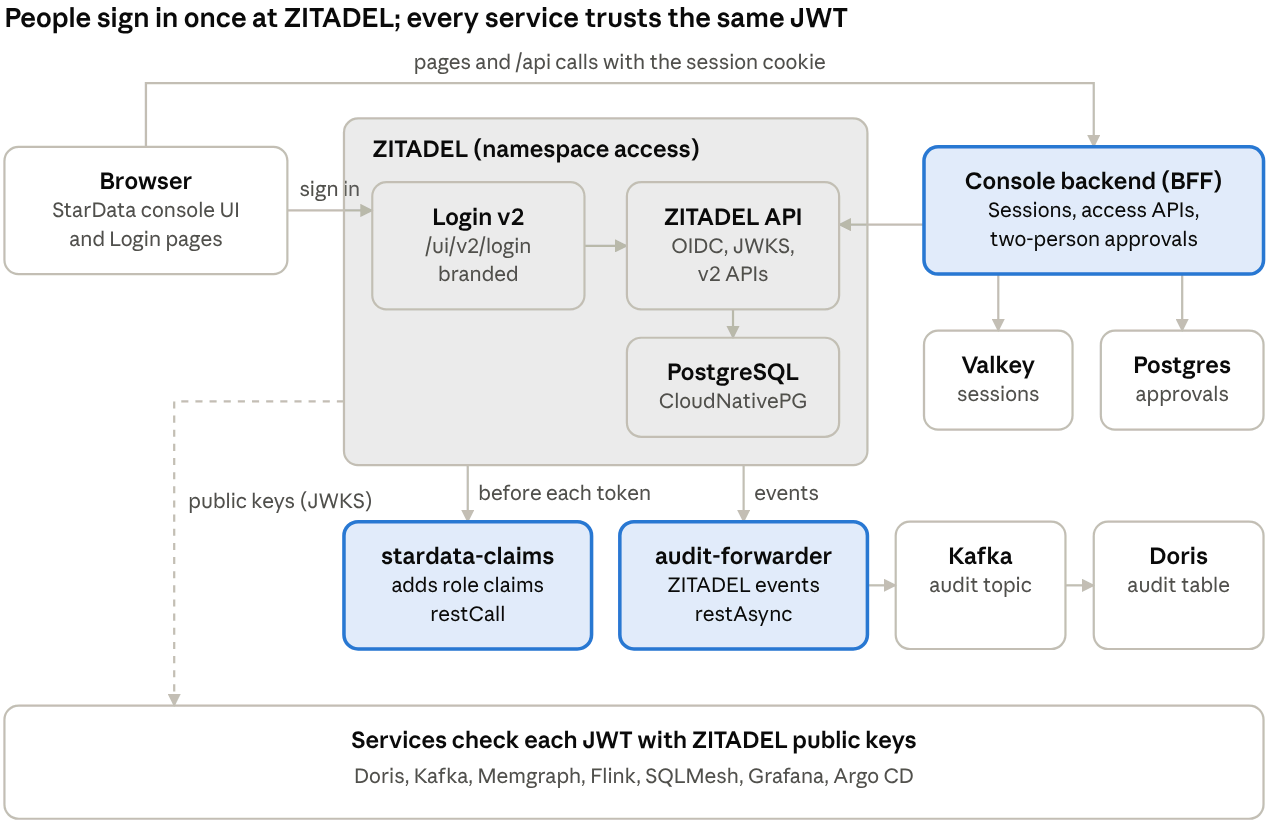
- ZITADEL v4 (API + Login v2) on on-premises Kubernetes with open-source PostgreSQL, deployed by Argo CD.
- All ZITADEL configuration as code (Terraform): organisation, project, roles, applications, machine users, sign-in, password, lockout and branding policies.
- `stardata-claims`: the Actions v2 target that adds `doris_role`, `kafka_role`, `memgraph_role`, `flink_role`, `sqlmesh_role`, `etl_role`, `mdm_role`, `monitoring_role` and `platform_roles` to access tokens.
- Console backend (BFF) sign-in, sessions and role checks, plus the access APIs behind Members, Member detail, Invite, Applications, Roles, Approvals, Sign-in policy and Audit.
- `audit-forwarder`: ZITADEL events into Kafka `platform.audit` and from there into Doris.

Fixed rules:

- One ZITADEL organisation (`stardatadb`) and one project (`stardata-platform`). Every role is a project role; a person holds at most one level per component.
- The browser never holds ZITADEL tokens. The console backend keeps them server-side and gives the browser an HttpOnly session cookie.
- Every service enforces its own role claim; hiding a button is never the only check.
- ZITADEL settings change only through Git (Terraform), except emergency use of the break-glass Owner in ZITADEL's own console.
- Changes to Owners and Admins need a second Owner (two-person rule).

Out of scope: single sign-on from an external company identity provider (can be added later as a ZITADEL identity provider), customer-facing sign-in, SAML applications.

Architecture



Highlighted boxes are built by the StarData team; ZITADEL is installed from its Helm chart.

access architecture · ZITADEL, console backend, claims and audit

The browser signs in on ZITADEL Login v2. Before each token, ZITADEL calls our `stardata-claims` service, which adds the component role claims. The console backend holds the session in Valkey, calls ZITADEL's v2 APIs with its own machine user for every access change, and keeps approvals in its own Postgres. ZITADEL events go through `audit-forwarder` to Kafka and into Doris; the backend writes its own audit events to the same topic. Every service checks the JWT against ZITADEL's public keys, so no service keeps its own user list.

Repository layout and tech stack

```
stardata-access/
  zitadel/
    helm/values-<env>.yaml      # ZITADEL chart values per environment
    terraform/                  # org, project, roles, apps, users, policies,
branding
      modules/project-roles/
      env/dev | staging | prod/
    bootstrap/                  # one-off job: Actions v2 targets and
executions
  claims/                       # stardata-claims service (Go)
  audit-forwarder/              # ZITADEL events -> Kafka (Go)
  console-backend/              # access routes inside the existing BFF
    internal/auth/              # OIDC, sessions, role middleware
    internal/access/            # members, approvals, applications, policy,
audit
    migrations/                 # console database tables
  web/src/pages/access/         # React pages for the 12 screens
  deploy/helm/stardata-access/  # chart for claims, audit-forwarder, backend
config
  deploy/argocd/
```

Layer	Choice	Why
Identity	ZITADEL v4 (Helm chart 9.x), Login v2	Current major release; Login v2 is deployed by default with the v9 chart
ZITADEL database	Open-source PostgreSQL 16, run by the CloudNativePG operator (3 instances)	ZITADEL supports PostgreSQL 14–18; managed backups
Configuration as code	Terraform with the `zitadel/zitadel` provider	Same settings in every environment; reviewed in pull requests
Claims service	Go, standard library HTTP	Small, fast; called on every token
Audit forwarder	Go, `franz-go` Kafka client	Async target for ZITADEL events
Console backend	Existing BFF (Go or Node.js, as already chosen)	Access routes are added to it

Layer	Choice	Why
Console database	PostgreSQL (the console's own schema)	Approvals, app ownership, invite metadata
Sessions	Valkey	Server-side console sessions
Frontend	React, TypeScript, TanStack Query, React Router	Same app as the rest of the console
Delivery	Helm, Argo CD, Kubernetes Secrets (encrypted at rest in etcd)	Same as every platform part

Step 1: deploy ZITADEL v4 on Kubernetes

ZITADEL runs as two containers: the main ZITADEL container (API, ZITADEL's own console, OIDC endpoints) and the Login container (Login v2 at `/ui/v2/login`). Both are stateless; all data is in PostgreSQL (ZITADEL Kubernetes docs). The chart runs an `init` job (schema) and a `setup` job (first instance) before the pods start.

1.1 Prerequisites

Item	Value
Kubernetes	1.30+ (on-premises cluster)
Database	Open-source PostgreSQL 16 as a CloudNativePG Cluster: 3 instances on different nodes, one synchronous replica, data on local or Ceph volumes, scheduled backups plus WAL archiving to the on-prem S3-compatible store (MinIO or Ceph RGW) kept 14 days; database `zitadel`; an admin user for the init job and an app user `zitadel`
Domain	`auth.stardatadb.com` (one domain serves API, OIDC and `/ui/v2/login`)
TLS	cert-manager certificate `zitadel-tls` signed by the company CA issuer; the ingress controller is reached through MetalLB or the existing on-prem load balancer

Item	Value
Ingress	Your NGINX or Traefik ingress controller. ZITADEL needs HTTP/2 between ingress and pods (h2c); the chart's ingress settings handle this for NGINX and Traefik
Masterkey	32 random characters, stored in Kubernetes Secret `zitadel-masterkey` (key `masterkey`). Never change it after install: it encrypts secrets in the database
SMTP	Company SMTP relay (for example Postfix or Exchange) credentials and sender `no-reply@stardatadb.com`

1.2 Helm values (excerpt)

Check every key against the `values.yaml` of the exact 9.x chart you pin; the shape below follows the chart's documented structure.

```
# zitadel/helm/values-prod.yaml
replicaCount: 3
zitadel:
  masterkeySecretName: zitadel-masterkey
  configmapConfig:
    ExternalDomain: auth.stardatadb.com
    ExternalPort: 443
    ExternalSecure: true
    TLS: { Enabled: false }            # TLS ends at the ingress
    Database:
      Postgres:
        Host: zitadel-db-rw.access.svc.cluster.local   # CloudNativePG read-
write service
        Port: 5432
        Database: zitadel
        MaxOpenConns: 20
        User:  { Username: zitadel,  SSL: { Mode: require } }
        Admin: { Username: zitadel_admin, SSL: { Mode: require } }
    FirstInstance:
      Org:
        Name: stardatadb
```

```
        Human:                          # the first Owner; change the
password at first sign-in
          UserName: owner@stardatadb.com
          Email: { Address: owner@stardatadb.com, Verified: true }
    DefaultInstance:
      LoginPolicy:
        AllowRegister: true
        ForceMFA: false                 # MFA for Owners/Admins is enforced
by the console (Step 4)
  configSecretName: zitadel-db-passwords   # Kubernetes Secret with
Database.Postgres.User.Password and Admin.Password
login:
  enabled: true
  replicaCount: 2
ingress:
  enabled: true
  className: nginx
  hosts: [{ host: auth.stardatadb.com, paths: [{ path: /, pathType: Prefix }]
}]
  tls: [{ secretName: zitadel-tls, hosts: [auth.stardatadb.com] }]
login:
  ingress:
    enabled: true
    className: nginx
    hosts: [{ host: auth.stardatadb.com, paths: [{ path: /ui/v2/login,
pathType: Prefix }] }]
    tls: [{ secretName: zitadel-tls, hosts: [auth.stardatadb.com] }]
resources:
  requests: { cpu: 500m, memory: 512Mi }
  limits:   { memory: 1Gi }
pdb: { enabled: true, minAvailable: 2 }
metrics: { enabled: true, serviceMonitor: { enabled: true } }
```

(Merge the two `login:` blocks into one in the real file; they are split here for reading.)

1.3 Install order

1. Create the CloudNativePG Cluster zitadel-db, the `zitadel` database and both users (Cluster manifest in Git, synced by Argo CD; CNPG creates the app user and its Secret).

2. Create the Kubernetes Secrets `zitadel-masterkey` and `zitadel-db-passwords` with kubectl create secret (not from Git); confirm both exist before installing the chart.
3. Add the Argo CD `Application` for the chart (`https://charts.zitadel.com`, chart `zitadel`, pinned 9.x version) with `values-<env>.yaml`. Sync.
4. Watch the `init` and `setup` jobs complete, then 3 ZITADEL pods and 2 Login pods become Ready.
5. Sign in at `https://auth.stardatadb.com/ui/console` as the first Owner, change the password, enrol a passkey.
6. Create the machine user `terraform-admin` (Instance or Org Owner) with a JSON key for Step 2; store the key as Kubernetes Secret terraform-admin-key in namespace access and in the CI pipeline's secret variables. This is the only manual configuration step.
7. Configure SMTP (Terraform in Step 2) and send a test email.

1.4 Operations

- **Upgrades:** bump the chart version in Git; the setup job migrates the database. Read ZITADEL release notes first, test on staging, take a CloudNativePG on-demand backup first.
- **Backups:** CloudNativePG ScheduledBackup (daily) and continuous WAL archiving to object storage, plus an on-demand Backup before each upgrade. Restore = a new CNPG Cluster bootstrapped from the backup (point in time if needed) + the same masterkey.
- **Signing keys:** ZITADEL rotates OIDC signing keys; services must fetch JWKS from `https://auth.stardatadb.com/oauth/v2/keys` and cache it for no more than 1 hour.

Step 2: ZITADEL configuration as code

Everything except the first Owner and the `terraform-admin` key is created by Terraform with the ZITADEL provider. Run `terraform plan` in CI on every pull request and `apply` after review; a nightly plan reports drift (shown on the Sign-in policy screen). Attribute names below follow the provider's documented resources; confirm them against the pinned provider version.

2.1 Organisation, project, roles

```
provider "zitadel" {
  domain           = "auth.stardatadb.com"
  insecure         = false
  port             = "443"
  jwt_profile_file = var.terraform_admin_key_file
}

data "zitadel_org" "stardatadb" { id = var.org_id }   # created by the first-
instance setup

resource "zitadel_project" "platform" {
  name                     = "stardata-platform"
  org_id                   = data.zitadel_org.stardatadb.id
  project_role_assertion   = true    # put roles in tokens
  project_role_check       = false   # people with no role must still sign in
to see "Waiting for approval"
  has_project_check        = false
}

locals {
  roles = {
    "platform.owner" = "Platform",   "platform.admin" = "Platform",
    "doris.viewer" = "Doris", "doris.analyst" = "Doris", "doris.engineer" =
"Doris", "doris.admin" = "Doris",
    "kafka.viewer" = "Kafka", "kafka.producer" = "Kafka", "kafka.engineer" =
"Kafka",
    "memgraph.viewer" = "Memgraph", "memgraph.engineer" = "Memgraph",
    "flink.viewer" = "Flink", "flink.engineer" = "Flink", "sqlmesh.engineer"
= "SQLMesh",
    "etl.viewer" = "ETL", "etl.developer" = "ETL", "etl.operator" = "ETL",
"etl.admin" = "ETL",
    "mdm.viewer" = "MDM", "mdm.steward" = "MDM", "mdm.admin" = "MDM",
    "monitoring.viewer" = "Monitoring", "monitoring.operator" = "Monitoring",
  }
}

resource "zitadel_project_role" "r" {
  for_each     = local.roles
  org_id       = data.zitadel_org.stardatadb.id
  project_id   = zitadel_project.platform.id
```

```
  role_key     = each.key
  display_name = each.key
  group        = each.value
}
```

2.2 Applications

Application	Type	Grant types	Redirect URIs	Access token
`stardata-console`	Web, confidential (client secret, basic auth), PKCE	authorization code, refresh token	`https://console.stardatadb.com/auth/callback` ; post-logout `https://console.stardatadb.com/`	**JWT** (needed so the preaccesstoken Action runs), 10 min
`stardata-gateway`	Web, confidential	authorization code, refresh token	`https://tools.stardatadb.com/oauth2/callback`	JWT
`stardata-grafana`	Web, confidential	authorization code	`https://grafana.stardatadb.com/login/generic_oauth`	JWT
`stardata-argocd`	Web, confidential	authorization code	`https://argocd.stardatadb.com/auth/callback`	JWT
`platform-cli`	Native, PKCE, no secret	authorization code, device code, refresh token	`http://127.0.0.1:8085/callback`	JWT

```
resource "zitadel_application_oidc" "console" {
  org_id                       = data.zitadel_org.stardatadb.id
  project_id                   = zitadel_project.platform.id
  name                         = "stardata-console"
  redirect_uris                =
["https://console.stardatadb.com/auth/callback"]
  post_logout_redirect_uris    = ["https://console.stardatadb.com/"]
  response_types               = ["OIDC_RESPONSE_TYPE_CODE"]
  grant_types                  = ["OIDC_GRANT_TYPE_AUTHORIZATION_CODE",
"OIDC_GRANT_TYPE_REFRESH_TOKEN"]
  app_type                     = "OIDC_APP_TYPE_WEB"
  auth_method_type             = "OIDC_AUTH_METHOD_TYPE_BASIC"
  access_token_type            = "OIDC_TOKEN_TYPE_JWT"
  access_token_role_assertion  = true
  id_token_role_assertion      = true
}
# client_id and client_secret outputs -> kubernetes_secret_v1 "console_oidc"
```

2.3 Machine users

Machine user	ZITADEL manager role	Used by	Credential
`terraform-admin`	Org Owner (and Instance Owner for instance policies)	Terraform in CI	JSON key, Kubernetes Secret
`console-backend`	Org user manager + authorization manager (the smallest manager roles that can manage users and authorizations; confirm names in your version)	Members, Invite, Applications APIs	JSON key (JWT profile)
`audit-reader`	Org owner viewer	audit-forwarder backfill	JSON key
`mdm-jobs`, app identities	none (only project roles)	Pipelines, apps	client secret or key

2.4 Policies and branding

Resource	Settings
`zitadel_login_policy`	allow register on; username/password allowed; passwordless (passkeys) allowed; external IdPs off; MFA prompts enabled; password check lifetime 7 days; MFA check lifetime 12 h; session lifetimes matching the console (1 day idle, 7 days max)
`zitadel_password_complexity_policy`	min length 12, number, symbol, upper and lower case
`zitadel_lockout_policy`	max password attempts 5
`zitadel_label_policy`	primary colour #3346C4, font Inter, StarData logo and icon (light and dark), hide "powered by" if your licence allows
`zitadel_smtp_config`	Company SMTP relay host and port, sender `no-reply@stardatadb.com`, TLS on

Resource	Settings
`zitadel_privacy_policy`	links to StarData terms and privacy notice (shown on sign-up)

2.5 Actions v2 targets and executions (bootstrap job)

If your provider version has no Actions v2 resources, a one-off Kubernetes Job (`zitadel/bootstrap/`) calls the API with the `terraform-admin` token. It is idempotent: it looks targets up by name first.

```
# Target for token claims (response is used, so it must be a call, not async)
curl -X POST https://auth.stardatadb.com/v2/actions/targets \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"name":"stardata-claims","restCall":{"interruptOnError":true},
       "endpoint":"http://stardata-
claims.access.svc.cluster.local/zitadel/claims","timeout":"3s"}'
# -> returns id and signingKey; store signingKey in Kubernetes Secret claims-
target-key

curl -X PUT https://auth.stardatadb.com/v2/actions/executions \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"condition":{"function":{"name":"preaccesstoken"}},"targets":["<claims
target id>"]}'
curl -X PUT https://auth.stardatadb.com/v2/actions/executions \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"condition":{"function":{"name":"preuserinfo"}},"targets":["<claims
target id>"]}'

# Async target for audit events (Step 7)
curl -X POST https://auth.stardatadb.com/v2/actions/targets ... \
  -d '{"name":"audit-forwarder","restAsync":{},"endpoint":"http://audit-
forwarder.access.svc.cluster.local/zitadel/events","timeout":"5s"}'
# execution with an event condition for the user group of events -> audit-
forwarder
```

The Actions v2 request and response shapes are in the Actions v2 usage guide. Instance must have the Actions feature enabled; the targets must be reachable from the ZITADEL pods (same cluster, so use cluster DNS names).

Step 3: claims service (`stardata-claims`)

ZITADEL puts project roles in a nested claim (`urn:zitadel:iam:org:project:roles`) that most tools cannot read. `stardata-claims` is an Actions v2 target: ZITADEL calls it just before it issues an access token (`preaccesstoken`) and before it answers userinfo (`preuserinfo`). It returns `append_claims` with one short claim per component (Actions v2 usage).

3.1 Contract

Input	JSON from ZITADEL: `function` , `user` , `userinfo` , `user_grants` (each with project and `roles`), `org` , `user_metadata`
Output	`{"append_claims": [{"key":"doris_role","value":"analyst"}, ...]}` ; `{}` for a user with no roles
Security	Verify the `ZITADEL-Signature` header with the target's signing key (see ZITADEL's "verify payload integrity" guide); reject anything else with 401
Latency	p99 under 50 ms; ZITADEL timeout is 3 s and the target interrupts on error, so a broken claims service blocks token issue
Pure function	No database, no network calls: it only maps the grants it receives

3.2 Mapping rules

- Only roles from project `stardata-platform` count.
- Per component (prefix before the dot) keep the highest level; order: `doris` viewer < analyst < engineer < admin; `kafka` viewer < producer < engineer; `memgraph` , `flink` viewer < engineer; `sqlmesh` engineer; `etl` viewer < developer < operator < admin; `mdm` viewer < steward < admin; `monitoring` viewer < operator.
- Claim name = `<component>_role` , value = level (`doris_role: "analyst"`).
- `platform.owner` and `platform.admin` go into an array `platform_roles` .

- A person should never hold two levels of one component; if they do, keep the highest and log a warning (the console prevents it).

3.3 Code (sketch)

```
// claims/main.go
var order = map[string][]string{
  "doris": {"viewer", "analyst", "engineer", "admin"}, "kafka": {"viewer",
"producer", "engineer"},
  "memgraph": {"viewer", "engineer"}, "flink": {"viewer", "engineer"},
"sqlmesh": {"engineer"},
  "etl": {"viewer", "developer", "operator", "admin"}, "mdm": {"viewer",
"steward", "admin"},
  "monitoring": {"viewer", "operator"},
}

type grant struct {
  ProjectID string   `json:"projectId"`
  Roles     []string `json:"roles"`
}
type req struct {
  Function   string  `json:"function"`
  UserGrants []grant `json:"user_grants"`
}
type claim struct {
  Key   string `json:"key"`
  Value any    `json:"value"`
}

func handle(w http.ResponseWriter, r *http.Request) {
  body, _ := io.ReadAll(io.LimitReader(r.Body, 1<<20))
  if !verifySignature(r.Header.Get("ZITADEL-Signature"), body, signingKey) {
// HMAC check per ZITADEL docs
    http.Error(w, "bad signature", http.StatusUnauthorized); return
  }
  var in req
  if err := json.Unmarshal(body, &in); err != nil { http.Error(w, "bad json",
400); return }

  best := map[string]int{}; platform := []string{}
  for _, g := range in.UserGrants {
    if g.ProjectID != projectID { continue }
```

```
    for _, role := range g.Roles {
      comp, level, ok := strings.Cut(role, ".")
      if !ok { continue }
      if comp == "platform" { platform = append(platform, role); continue }
      if i := slices.Index(order[comp], level); i >= 0 && i+1 > best[comp] {
best[comp] = i + 1 }
    }
  }
  out := []claim{}
  for comp, n := range best { out = append(out, claim{comp + "_role",
order[comp][n-1]}) }
  out = append(out, claim{"platform_roles", platform})
  json.NewEncoder(w).Encode(map[string]any{"append_claims": out})
}
```

The JSON field names above follow the documented payload; log one real request in dev (with personal data masked) and adjust the structs before release.

3.4 Run it

Deployment in namespace `access`, 2 replicas, 50m CPU and 64 Mi each, readiness on `/healthz`, PodDisruptionBudget `minAvailable: 1`. NetworkPolicy: only ZITADEL pods may call it. Signing key from Kubernetes Secret claims-target-key; rotating it means creating a new target key in ZITADEL and rolling the Deployment.

3.5 Known limits

- `preaccesstoken` runs only for **JWT** access tokens; every application in Step 2 is set to JWT for this reason.
- Claims appended by the Action are in the JWT but not in token introspection responses, so services must validate the JWT locally (JWKS), not introspect it.
- Actions run for the user's organisation. StarData has one organisation, so no other organisation can inject roles; keep it that way.

Step 4: console backend sign-in

The console backend (BFF) is a confidential OIDC client. The browser only ever sees a session cookie.

4.1 Flow

1. `GET /auth/login?next=/members`: create `state`, `nonce` and a PKCE `code_verifier`; store them in Valkey under a short-lived pre-session key (5 min); redirect to `https://auth.stardatadb.com/oauth/v2/authorize` with `client_id`, `redirect_uri`, `response_type=code`, `scope=openid profile email offline_access urn:zitadel:iam:org:project:id:<project_id>:aud`, `code_challenge`, `code_challenge_method=S256`, `state`, `nonce`.
2. ZITADEL Login v2 handles password, passkey, MFA and sign-up.
3. `GET /auth/callback?code&state`: check `state`; exchange the code at `/oauth/v2/token` with client secret and `code_verifier`; validate the ID token (signature via JWKS, `iss`, `aud`, `exp`, `nonce`).
4. Create the session: random 256-bit session ID; Valkey hash `sess:<id>` = `{sub, email, name, access_token, refresh_token, id_token, access_expires_at, mfa, created_at}`; TTL = idle limit (24 h), refreshed on each request; absolute limit 7 days kept in the hash.
5. Set cookie `__Host-stardata_session=<id>; Path=/; Secure; HttpOnly; SameSite=Lax`.
6. Redirect to `next` (only relative paths allowed).

4.2 Every request

- Look up the session; if missing or past the absolute limit → 401 (the frontend sends the person to `/auth/login`).
- If the access token expires within 60 seconds, refresh it with the refresh token (one refresh at a time per session: Valkey lock `sess:<id>:refresh`, 5 s). If refresh fails because the authorization was removed or the user deactivated → delete the session, 401.
- Decode the access token's claims once per token and cache them in the request context: `platform_roles`, `*_role`, `amr` (sign-in methods).
- Role middleware per route, for example `RequireAny("platform.owner", "platform.admin")` for Members.
- **MFA for Owners and Admins:** if `platform_roles` is not empty and `amr` does not include an MFA method (`mfa`, `otp`, `user` presence with passkey), return 403 `mfa_required`; the frontend sends the person through `/auth/login?prompt=login&acr=mfa` to sign in again with MFA. (Confirm the exact `amr` values ZITADEL returns in dev and adjust.)

- **Waiting for approval:** a signed-in person with no `*_role` claims and no `platform_roles` gets `{"status":"waiting"}` from `GET /api/me`; the frontend shows the Waiting screen.
- **CSRF:** every state-changing request needs header `X-CSRF-Token` matching the value stored in the session (double-submit). SameSite=Lax already blocks most cross-site posts.

4.3 Sign-out

`POST /auth/logout`: delete `sess:<id>` in Valkey, clear the cookie, return the URL `https://auth.stardatadb.com/oidc/v1/end_session?id_token_hint=<id_token>&post_logout_redirect_uri=https://console.stardatadb.com/`, which the frontend opens. This ends the ZITADEL session too.

4.4 `GET /api/me`

```
{
  "sub": "284019333120", "name": "Priya Shah", "email":
"priya@stardatadb.com",
  "status": "active",
  "platform_roles": [],
  "roles": {"doris": "analyst", "kafka": "viewer", "etl": "viewer", "mdm":
"steward", "monitoring": "viewer"},
  "mfa": true, "session_expires_at": "2026-11-13T09:40:00Z", "csrf": "
<token>"
}
```

The sidebar, tiles and Administration group are drawn from this response.

Step 5: access APIs

The console backend calls ZITADEL with the `console-backend` machine user (JWT profile, its own token cached until expiry), never with the signed-in person's token. It checks the person's role first, then acts. Use ZITADEL's v2 services: User, Session, Settings, Project and **Authorization** (role assignments). The v1 Management "user grant" calls are deprecated in favour of the Authorization service; one community report shows an early v4 release returning 404 for `CreateAuthorization`, so test it on the pinned version and keep the v1 call as a fallback behind a flag. Check every method name below against the ZITADEL API reference for your version.

5.1 Endpoints and the ZITADEL calls behind them

Endpoint	Screen	Allowed	ZITADEL calls	Console DB
`GET /api/access/members?state=waiting`	Members	Owner, Admin	UserService ListUsers (humans, email verified) minus users with an authorization on the project	—
`GET /api/access/members?state=active&q=`	Members	Owner, Admin	AuthorizationService ListAuthorizations (project) + UserService ListUsers	—
`GET /api/access/members/{id}`	Member detail	Owner, Admin	UserService GetUserByID; ListAuthorizations (user); SessionService ListSessions (user); auth methods	audit rows from Doris
`POST /api/access/members/{id}/approve`	Members	Owner, Admin	AuthorizationService CreateAuthorization (project, role keys)	approval row if levels include `platform.*`
`POST /api/access/members/{id}/decline`	Members	Owner, Admin	UserService DeactivateUser; email via the SMTP relay	decision row
`PUT /api/access/members/{id}/levels`	Member detail	Owner, Admin	AuthorizationService UpdateAuthorization (replace role keys)	approval row for `platform.*` changes
`POST /api/access/members/{id}/revoke`	Member detail	Owner, Admin	DeleteAuthorization; ListSessions + DeleteSession for each	delete Valkey sessions; Doris proxy revoke
`DELETE /api/access/members/{id}/sessions/{sid}`	Member detail	Owner, Admin	SessionService DeleteSession	—
`POST /api/access/members/{id}/reset-mfa`	Member detail	Owner, Admin	UserService remove TOTP / passkeys / U2F methods	—
`POST /api/access/members/{id}/password-reset`	Member detail	Owner, Admin	UserService PasswordReset (ZITADEL sends the email)	—
`POST /api/access/members/invite`	Invite	Owner, Admin	UserService AddHumanUser + CreateInviteCode (ZITADEL sends the invite); CreateAuthorization	invite row
`GET /api/access/apps`	Applications	Owner, Admin	UserService ListUsers (machine) + ListAuthorizations	`app_identity` (owner, expiry, credential type)

Endpoint	Screen	Allowed	ZITADEL calls	Console DB
`POST /api/access/apps`	Applications, App secret	Owner, Admin	UserService create machine user; add secret or key; CreateAuthorization (data roles only, no engineer levels)	`app_identity`
`POST /api/access/apps/{id}/rotate`	Applications	Owner, Admin	add new secret or key; schedule removal of the old one after 24 h	`app_identity.rotation_due`
`POST /api/access/apps/{id}/revoke`	Applications	Owner, Admin	remove secrets and keys; DeleteAuthorization; DeactivateUser	—
`GET /api/access/roles`	Roles	Owner, Admin	ProjectService list project roles; counts from ListAuthorizations	—
`GET /api/access/approvals`, `POST /api/access/approvals/{id}`	Approvals	Owner (Admin may view)	runs the stored change on approve	`approval`
`GET /api/access/policy`	Sign-in policy	Owner	SettingsService: login, password complexity, lockout, branding settings	Terraform drift status from CI
`GET /api/access/audit?who=&component=&result=&from=&to=`	Audit	Owner, Admin	—	Doris `platform_audit` (read-only user)
`GET /api/access/summary`	Sidebar badges	Owner, Admin	same as the waiting members list, count only	pending approvals count
`GET /api/access/audit/export`	Audit	Owner, Admin	—	Doris platform_audit, streamed CSV up to 100,000 rows; the export is audited

5.2 Business rules enforced in the backend

- One level per component: the backend turns the level pickers into role keys and refuses two keys with the same prefix.
- Admins can change data levels but cannot add, remove or change `platform.owner` / `platform.admin`, and cannot change an Owner.
- Adding or removing `platform.*` creates an `approval`; the change runs only when a different Owner approves within 1 hour.
- Never fewer than two Owners: revoke, decline or demote of an Owner is refused if it would leave one.

- Nobody changes their own platform role.
- Applications never get engineer or admin levels, never `platform.*`.
- Every write sends an audit event (Step 7) with before and after values.

5.3 Console database (PostgreSQL)

```
CREATE TABLE approval (
  id            text PRIMARY KEY,               -- A-1029
  action        text NOT NULL,                  -- set_platform_role |
remove_platform_role
  target_user   text NOT NULL,                  -- ZITADEL user ID
  payload       jsonb NOT NULL,                 -- {"from":[...],"to":[...]}
  reason        text,
  requested_by  text NOT NULL,
  requested_at  timestamptz NOT NULL DEFAULT now(),
  expires_at    timestamptz NOT NULL,
  state         text NOT NULL DEFAULT 'pending', -- pending | approved |
rejected | expired
  decided_by    text,
  decided_at    timestamptz,
  CHECK (decided_by IS NULL OR decided_by <> requested_by)
);

CREATE TABLE app_identity (
  user_id          text PRIMARY KEY,            -- ZITADEL machine user ID
  name             text UNIQUE NOT NULL,
  owner_user_id    text NOT NULL,
  credential_type  text NOT NULL,               -- client_secret | jwt_key |
client_cert | app_password
  expires_at       timestamptz,
  rotation_due     timestamptz,
  created_at       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE invite (
  user_id     text PRIMARY KEY,
  email       text NOT NULL,
  invited_by  text NOT NULL,
```

```
  levels      jsonb NOT NULL,
  sent_at     timestamptz NOT NULL DEFAULT now(),
  accepted_at timestamptz
);
```

A CronJob every 5 minutes expires approvals and sends expiry reminders for app credentials (14, 7 and 1 days).

Step 6: frontend screens

The access pages live in the main StarData console (React + TypeScript, under `web/src/pages/access`). They appear in the sidebar group **Access (ZITADEL)** only when `/api/me` says the person is Owner or Admin. Sign-in, sign-up and the MFA screens are not built by us: they are ZITADEL Login v2 with our branding. Each screen below matches one board on the access canvas.

Canvas board	Route	Built by	Main API calls	Notes
1 Sign in and MFA	`/ui/v2/login` on auth.stardatadb.com	ZITADEL Login v2	OIDC authorize	Branding from the label policy (logo, colours, font)
2 Create account and verify email	`/ui/v2/login/register`	ZITADEL Login v2	—	Self-registration on; email domain allow-list in login settings
3 Waiting for approval	`/waiting`	Console	`GET /api/me` (no roles yet)	Poll every 30 s; shows the request time and who approves

Canvas board	Route	Built by	Main API calls	Notes
4 Members	`/access/members`	Console	members list (waiting, active, deactivated); approve; decline	Tabs with counts; badge in the sidebar shows waiting count
5 Member detail	`/access/members/:id`	Console	member detail; levels; sessions; revoke; reset MFA	Level pickers, one per component; Revoke now asks to type the name
6 Invite a person	`/access/members/invite`	Console (dialog)	invite	Email, name, levels, optional message
7 Applications	`/access/apps`	Console	apps list; create; rotate; revoke	Expiry pill turns amber at 14 days, red at 7
8 Application created	`/access/apps/:id/created`	Console (dialog)	response of create	Secret shown once, copy button, download as .env; never stored in the browser
9 Roles and token claims	`/access/roles`	Console	roles	Read-only; shows the claim each level produces

Canvas board	Route	Built by	Main API calls	Notes
10 Approvals	`/access/approvals`	Console	approvals list; approve; reject	Approve button disabled for the requester
11 Sign-in policy	`/access/policy`	Console	policy	Read-only view; Edit opens a link to the Terraform repo
12 Audit	`/access/audit`	Console	audit search; event detail drawer	Filters: who, component, action, result, time; export CSV

Frontend rules:

- Use TanStack Query for all calls; invalidate the member and approval lists after every write.
- Send the CSRF header (`X-CSRF-Token` from the `csrf` cookie) on every POST, PUT and DELETE.
- On 401, redirect to `/auth/login?return=<path>` ; on 403 show "You don't have access to this page" with the needed role.
- Hide buttons the person can't use, but the backend still checks everything.
- Destructive actions (decline, revoke, rotate) use a confirm dialog with the reason field required.

Step 6b: how to build each screen

This section is the build spec for each board on the access canvas. Build the shared parts first, then the screens in the order below. All JSON is the console backend's own API (Step 5), not ZITADEL's.

Shared parts (build first)

Part	What it does	Build notes
`AccessLayout`	Topbar, sidebar group Access (ZITADEL) with badges	Badges from `GET /api/access/summary` returning waiting count and pending approvals; refetch every 60 s
`RequireRole`	Route guard	Reads `/api/me`; Owner or Admin for all access pages, Owner only for Sign-in policy; else 403 page
`LevelPicker`	One segmented control per component: None plus its levels	Levels come from `GET /api/access/roles`, never hard-coded; returns one role key or null; `disabledLevels` prop for apps
`LevelGrid`	All pickers for one person, plus Platform (Member, Admin, Owner)	Platform shows a note "needs a second Owner" when Admin or Owner is picked
`ClaimsPreview`	Shows the token claims the chosen levels will produce	Pure function: role key `doris.analyst` gives `doris_role=analyst`; platform keys go into the `platform_roles` array
`ConfirmDialog`	Confirm a destructive action	Reason field required; for Revoke now the person must type the member's name
`DataTable`	Tables with sort, filter, paging	TanStack Table; server-side paging with a cursor
`Pill`, `Drawer`, `CopyField`, `Toast`	Status labels, side panel, copy-to-clipboard, messages	`CopyField` never writes the value to local storage or logs

Shared types (`web/src/pages/access/types.ts`):

```
export type Component =
'doris'|'kafka'|'memgraph'|'flink'|'sqlmesh'|'etl'|'mdm'|'monitoring';
export type Levels = Partial<Record<Component, string>>;       //
{doris:'analyst', kafka:'viewer'}
export type PlatformRole = 'member'|'admin'|'owner';
export interface Member {
  id: string; name: string; email: string; state:
'waiting'|'active'|'deactivated';
  emailVerified: boolean; personalDomain: boolean; createdAt: string;
  platform: PlatformRole; levels: Levels; lastSignIn?: string;
}
export interface RoleCatalogueItem {
  component: Component|'platform'; prefix: string; levels: string[];
  claim: string; allows: string; counts: number[];
}
```

1. Sign in and MFA (ZITADEL Login v2)

- **Built by ZITADEL**, not by us. We only set branding and policy in Terraform (Step 2).
- Branding: `zitadel_label_policy` with logo, icon, primary colour #3346C4, font Inter, "hide login name suffix" on; upload logo and font files with the label policy resources, then activate.
- Screen 1: email, then password, "Sign in with a passkey", "Forgot password?", "Create account". Screen 2: six-digit TOTP, passkey or security key, recovery code.
- MFA rule: Owners and Admins must have MFA. ZITADEL's login policy can't force MFA per role, so the BFF checks the `amr` claim after sign-in; if the person is Owner or Admin and `amr` has no `mfa`, `otp`, `hwk` or `swk`, it sends them to `/auth/login?prompt=login&acr_values=...` again and shows "Set up two-step verification" with a link to ZITADEL's self-service MFA setup. Check the exact `amr` values your version issues.
- Lockout after 5 wrong passwords (lockout policy).
- Test: wrong password 5 times locks; an Owner without MFA cannot reach any console page.

2. Create account and verify email (ZITADEL Login v2)

- Built by ZITADEL. Turn on self-registration in the login policy; password complexity 12+, number, symbol.

- Terms and privacy links: set in the ZITADEL privacy policy (`zitadel_privacy_policy` with `tos_link` and `privacy_link`).
- The verification code email uses the ZITADEL message texts; customise the "verify email" text with our wording.
- After verification ZITADEL redirects back to the console; the person has no roles, so the BFF sends them to `/waiting` .
- Email domain check: the backend flags (does not block) addresses outside the allow-list `stardatadb.com` ; the flag shows in Members.

3. Waiting for approval (`/waiting`)

- Layout: centred card; three-step timeline (Account created, Email verified, Admin gives you access); buttons Check again and Sign out; a note about wrong email.
- Data: `GET /api/me` returns `status: "waiting"` , `email` , `emailVerifiedAt` .
- Behaviour: poll `/api/me` every 30 s and on Check again; when `status` becomes `active` , force a token refresh (`POST /auth/refresh`) and go to Home.
- When someone signs up, the backend emails Owners and Admins (one email per sign-up, rate-limited to one per 10 min).
- Any other route for a waiting user returns 403 from the backend and redirects here.

4. Members (`/access/members`)

- Layout: header with Invite by email; section "Waiting for approval" with one card per person (avatar initials, name, email, verified, signed up time, a `LevelGrid` , Approve with these roles, Decline); warning line when `personalDomain` is true; then "All members" table with search (Person, Platform, Doris, Kafka, Memgraph, ETL Studio, Last sign-in, Edit).
- Data:

```
GET /api/access/members?state=waiting
GET /api/access/members?state=active&q=priya&cursor=...
```

```
{"items":[{"id":"2911...","name":"Ana
Torres","email":"ana@stardatadb.com","state":"waiting","emailVerified":true,"
personalDomain":false,"createdAt":"2026-11-
06T07:40:00Z","platform":"member","levels":{}}],"next":null}
```

- Approve:

```
POST /api/access/members/2911.../approve
{"platform":"member","levels":{"doris":"viewer","etl":"viewer"}}
-> 200 {"state":"active"}   or   202 {"approvalId":"A-1030"} when platform is
admin or owner
```

- Validation: at least one level or a platform role, else the button is disabled; the backend rejects two keys with the same prefix and rejects levels unknown to the catalogue.
- Decline opens `ConfirmDialog` (reason required), calls `POST .../decline`.
- After success: remove the card, toast "Ana Torres can now sign in with these roles", refresh the sidebar badge.
- States: loading skeleton; empty waiting list shows "No one is waiting"; errors show the backend message.

5. Member detail (`/access/members/:id`)

- Layout: header (name, email, member since, ZITADEL user ID, state pill, Reset MFA, Send password reset, Revoke now); card Access levels (`LevelGrid` with MDM and Monitoring too, Save changes, Cancel, `ClaimsPreview`); card Active sessions (Where, IP, Signed in, Method, End; End all sessions); card Sign-in and change history (last 10 audit rows, Open in Audit).
- Data: `GET /api/access/members/:id` returns the member plus `sessions[]` (`id` , `userAgent` , `ip` , `createdAt` , `method`) and `history[]` (from Doris).
- Save: `PUT /api/access/members/:id/levels` with the full new set; the response says `applied` or `pendingApproval` with the approval ID; show the right toast.
- Revoke now: `ConfirmDialog` asks to type the full name and a reason; then `POST .../revoke` . The backend deletes the authorization, ends ZITADEL sessions, deletes Valkey sessions, and asks the Doris proxy to drop app passwords for that user.
- Disable all buttons for your own account's platform role, and for Owners when you are an Admin.
- Note on timing: a changed level reaches the person's next token; existing tokens stay valid until they expire (10 minutes). Revoke now also ends sessions, so it takes effect at once in the console.

6. Invite a person (dialog on Members)

- Fields: work email (required, valid email), first name, last name, `LevelGrid`, optional message (max 500 characters).
- `POST /api/access/members/invite` with `{email, firstName, lastName, platform, levels, message}`. The backend creates the user in ZITADEL, asks ZITADEL to send the invite code email, and stores the levels; when the person finishes setup, the levels are applied (no waiting screen).
- Errors: 409 when the email already exists (show "Open member" link).

7. Applications (`/access/apps`)

- Layout: table (Application name and description, Data roles, Credential, Status, Owner, Rotate, Revoke) and a "New application" panel.
- Status pill: green normally, amber when the credential expires within 14 days, red within 7.
- New application form: Name (lowercase, numbers, dashes; 3 to 40 characters; unique), Owner (a person), Data roles (checkboxes from the catalogue, engineer and admin levels hidden), How it signs in (Client certificate, Client ID and secret, App password).
- Credential types and how each is made:

Choice	Used for	How the backend makes it
Client certificate	Doris from your own code	Issue a 90-day client certificate from the platform CA (cert-manager CA issuer) with the app name as CN; the Doris proxy maps CN to the app
Client ID and secret	Kafka OAUTHBEARER, Memgraph, APIs	ZITADEL machine user plus client secret; services get short-lived JWTs with client credentials
App password	Doris only, for BI tools that need a user and password	Generated in the Doris proxy, 90 days, hashed there; tied to the ZITADEL machine user

- Rotate: creates a new credential, shows it once (screen 8), keeps the old one 24 hours. Revoke: `ConfirmDialog`, removes all credentials and the authorization at once.

8. Application created (dialog)

- Shows client ID, client secret (or certificate and key, or app password), Copy buttons, Download as .env, and the warning "shown once".
- The create or rotate response is the only place the secret appears; keep it in component state only, clear it when the dialog closes, never put it in TanStack Query's cache (use a mutation result, not a query).
- Closing asks "Have you saved the secret?".

9. Roles and token claims (`/access/roles`)

- Read-only table from `GET /api/access/roles` (catalogue plus people per level). Badge "Defined in Git (Terraform)".
- The backend builds the catalogue from ZITADEL project roles; role key format `component.level`, display name and group set in Terraform.
- Section "How a role reaches a service": a static five-step explanation.

10. Approvals (`/access/approvals`)

- Layout: stat tiles (Waiting for you, Owners with "minimum 2 always kept", Admins); table (ID, Change, Asked by, When, Expires, Approve, Reject); detail panel (before and after levels, reason, checks: MFA enrolled, signed in within 30 days).
- Data: `GET /api/access/approvals?state=pending|all`. Decide: `POST /api/access/approvals/:id` with `{decision:"approve"|"reject", comment}`.
- The Approve button is hidden for the requester and for Admins; the backend also refuses (`409 same_person`).
- Expiry shown as a countdown; expired rows go grey.

11. Sign-in policy (`/access/policy`)

- Owner only. Stat tiles: ZITADEL pods, Login v2 pods, Database (CloudNativePG cluster health), Config drift (from the last CI `terraform plan`), Signing keys.
- Data: `GET /api/access/policy` reads ZITADEL settings (login, password complexity, lockout, label, SMTP) and pod health from the Kubernetes API (read-only ServiceAccount on namespace `access`), drift from a JSON file CI writes to object storage.
- "Open change in Git" links to the Terraform folder in the repo. Nothing on this page writes to ZITADEL.

- Break-glass note: static text.

12. Audit (`/access/audit`)

- Layout: filters (Person or app, Component, Result, Time), Export CSV, table (Time, Who, Component, Action, Source IP, Result), Load more, and an event detail drawer with all fields and the raw JSON.
- Data: `GET /api/access/audit?who=&component=&result=&from=&to=&cursor=` runs a parameterised query on Doris `platform_audit` (read-only user), newest first, 50 rows a page.
- Export CSV: `GET /api/access/audit/export?...` streams up to 100,000 rows; the export itself is audited.
- The drawer shows Trace ID with a link to the trace in Grafana when present.

Step 7: audit trail

Two sources feed one table, so the Audit screen shows everything in one place.

1. **ZITADEL events.** An Actions v2 execution on event conditions (user, session, authorization, project role, machine secret events) calls the `audit-forwarder` target as `restAsync`. The forwarder checks the signature, maps the event to the common shape and writes it to Kafka topic `platform.audit`.
2. **Console events.** The BFF writes its own events (approval requested, approved, rejected; app created; rotation; revoke reason; access denied) straight to `platform.audit` with the same shape.

Common event shape (JSON, key = `event_id`):

```
{
  "event_id": "01JC9V...",
  "ts": "2026-11-06T10:42:13Z",
  "source": "zitadel",
  "actor_id": "2840...", "actor_name": "Priya Nair",
  "target_type": "user", "target_id": "2911...", "target_name": "Sam Lee",
  "component": "doris",
  "action": "authorization.changed",
  "before": ["doris.viewer"], "after": ["doris.analyst"],
  "result": "ok",
  "reason": "Joins the finance analytics team",
  "ip": "10.20.4.17", "user_agent": "Mozilla/5.0 ...",
  "request_id": "b7f2..."
}
```

Doris table and load:

```
CREATE TABLE platform.platform_audit (
  ts DATETIME(3), event_id VARCHAR(40), source VARCHAR(16),
  actor_id VARCHAR(40), actor_name VARCHAR(200),
  target_type VARCHAR(20), target_id VARCHAR(40), target_name VARCHAR(200),
  component VARCHAR(20), action VARCHAR(60),
  before_value JSON, after_value JSON, result VARCHAR(10), reason STRING,
  ip VARCHAR(45), user_agent STRING, request_id VARCHAR(40)
) DUPLICATE KEY (ts, event_id)
PARTITION BY RANGE (ts) ()
DISTRIBUTED BY HASH (event_id) BUCKETS 8
PROPERTIES ("dynamic_partition.enable"="true",
"dynamic_partition.time_unit"="DAY",
            "dynamic_partition.start"="-10", "dynamic_partition.end"="3");

CREATE ROUTINE LOAD platform.audit_load ON platform_audit
PROPERTIES ("format"="json", "jsonpaths"="[...]")
FROM KAFKA ("kafka_broker_list"="kafka-bootstrap:9093",
"kafka_topic"="platform.audit");
```

Rules: keep 10 days (Doris drops partitions older than 10 days); the console reads with a read-only Doris user; nobody can delete rows from the console; the forwarder retries with backoff and sends failures to `platform.audit.dlq`. Doris copies the table to Iceberg itself if you want it in the lakehouse; nothing else writes there.

Step 8: Kubernetes deployment

Everything for access runs in namespace `access` and is deployed by Argo CD from `stardata-access/deploy`.

Workload	Kind	Replicas	Notes
zitadel	Helm chart (Deployment + init and setup Jobs)	3	PDB minAvailable 2; anti-affinity across zones
zitadel-login	part of the chart	2	Login v2 UI
stardata-claims	Deployment + Service (ClusterIP)	2	Called by ZITADEL only; p99 under 50 ms
audit-forwarder	Deployment + Service (ClusterIP)	2	Writes to Kafka with SASL/SCRAM or mTLS
console-backend	Deployment (in `console` namespace)	3	Holds the access endpoints
access-cron	CronJob	—	Approval expiry, credential reminders
zitadel-db	CloudNativePG Cluster (PostgreSQL 16)	3	Open-source operator; services zitadel-db-rw and zitadel-db-ro; ScheduledBackup and WAL archive to MinIO or Ceph RGW. The console DB is a second small Cluster, console-db

Argo CD sync order (sync waves): 0 NetworkPolicies (Secrets are created before the first sync and are not in Git), 1 ZITADEL chart, 2 Terraform job (or CI apply), 3 claims and audit-forwarder, 4 Actions bootstrap job, 5 console-backend.

Example for the claims service:

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: stardata-claims, namespace: access }
spec:
  replicas: 2
  selector: { matchLabels: { app: stardata-claims } }
  template:
```

```
    metadata: { labels: { app: stardata-claims } }
    spec:
      securityContext: { runAsNonRoot: true, seccompProfile: { type:
RuntimeDefault } }
      containers:
        - name: claims
          image: ghcr.io/stardatadb/stardata-claims:1.0.0
          ports: [{ containerPort: 8080 }]
          env:
            - name: SIGNING_KEY
              valueFrom: { secretKeyRef: { name: claims-target-key, key:
signingKey } }
          readinessProbe: { httpGet: { path: /healthz, port: 8080 } }
          resources: { requests: { cpu: 100m, memory: 64Mi }, limits: {
memory: 128Mi } }
          securityContext: { readOnlyRootFilesystem: true,
allowPrivilegeEscalation: false }
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: claims-from-zitadel-only, namespace: access }
spec:
  podSelector: { matchLabels: { app: stardata-claims } }
  ingress:
    - from: [{ podSelector: { matchLabels: { app.kubernetes.io/name: zitadel
} } }]
      ports: [{ port: 8080 }]
```

Secrets (Kubernetes Secrets in namespaces access and console, created by the platform team, never committed to Git): ZITADEL masterkey, DB passwords, SMTP, claims and audit target signing keys, console-backend machine key, BFF OIDC client secret, Valkey password, Kafka credentials. Default-deny NetworkPolicy in `access`; allow ingress-nginx to zitadel, zitadel to claims and audit-forwarder, console-backend to zitadel, audit-forwarder to Kafka.

Security checklist

- [ ] ZITADEL masterkey is 32 characters, stored only in Kubernetes Secret zitadel-masterkey, backed up offline; losing it means losing encrypted data.
- [ ] Encryption at rest for Secrets turned on in the API server (EncryptionConfiguration with aescbc or a KMS plugin); check with `etcdctl` that values are not plain text.

- [ ] Kubernetes RBAC: only the platform team and each workload's own ServiceAccount can `get` Secrets in `access` and `console`; no `list` or `watch` on Secrets for anyone else.
- [ ] No Secret values in Git, Helm values or Argo CD apps; charts only reference Secrets by name. Keep an offline encrypted copy of every Secret for disaster recovery.
- [ ] Rotation is manual for now: update the Secret, then `kubectl rollout restart` the Deployment; record each rotation in the audit log.
- [ ] TLS everywhere: load balancer and ingress to ZITADEL, PostgreSQL (CNPG) with `sslmode=verify-full`, Kafka with TLS.
- [ ] MFA required for Owners and Admins (login policy) and checked again in the BFF through the `amr` claim.
- [ ] Passkeys allowed; SMS OTP off.
- [ ] Lockout after 5 wrong passwords; password policy 12+ characters.
- [ ] Access tokens 10 minutes, refresh tokens 12 hours idle, 7 days max.
- [ ] Every target call checked with the ZITADEL-Signature HMAC; reject if older than 5 minutes.
- [ ] Claims service fails closed: on error ZITADEL issues no extra claims, so services treat the user as having no role.
- [ ] Session cookie `__Host-`, Secure, HttpOnly, SameSite=Lax; CSRF double-submit on writes.
- [ ] Two-person rule and minimum two Owners enforced in the backend, not only the UI.
- [ ] Break-glass Owner account: hardware key, stored in a safe, use triggers an alert.
- [ ] ZITADEL's own console (`/ui/console`) limited by IP allow-list to the platform team VPN.
- [ ] Application secrets shown once; rotation at most 90 days; reminders at 14, 7 and 1 days.
- [ ] Rate limit sign-in and the access API at the ingress.
- [ ] Containers non-root, read-only root filesystem; images scanned in CI (Trivy) and signed (cosign).
- [ ] Audit rows kept 10 days and not deletable from the console.

Monitoring and alerts

ZITADEL exposes Prometheus metrics (in v4 at `/debug/metrics`; check the path for your version). Add ServiceMonitors for ZITADEL, the claims service, the audit-forwarder and the console backend, and a Grafana dashboard "Access" in the Monitoring console.

Signal	Source	Alert
Sign-in success and failure rate	ZITADEL metrics and audit table	Failures over 20% for 10 min
Locked accounts	audit table	Over 5 in 15 min
Claims target latency and errors	`claims_request_duration_seconds`, `claims_errors_total`	p99 over 200 ms or any errors for 5 min
Audit forwarder lag	Kafka consumer lag, DLQ size	DLQ over 0
ZITADEL pods and DB	kube-state-metrics, CloudNativePG metrics	Pods under 2 ready; DB CPU over 80%
Token signing key age	ZITADEL web keys	Older than 90 days
App credentials near expiry	console DB	Under 7 days
Pending approvals	console DB	Older than 45 min
Break-glass sign-in	audit table	Any

Logs: JSON to stdout, collected by the cluster log agent; never log tokens, secrets or passwords.

Testing and acceptance

Automated: Go unit tests for the claims mapping (every role combination, highest level wins, unknown role ignored, bad signature refused); contract test that runs a real ZITADEL in Docker Compose, signs in a test user and checks the JWT claims; Playwright tests for every console screen; k6 load test at 50 sign-ins per second with claims p99 under 50 ms.

Screen	Acceptance check
Sign in	Branding shows; wrong password 5 times locks the account; Owner without MFA is asked to set it up
Create account	Verification email arrives; disallowed email domain is refused
Waiting	A new user with no roles lands here and can't open any other page
Members	Approve gives the chosen levels; the user's next token has the right claims
Member detail	Changing a level shows in a new token within one refresh; Revoke now ends all sessions in under a minute
Invite	Invite email arrives; accepting it skips the waiting screen
Applications	Machine user can get a token with client credentials; revoke stops it at once
App secret	Secret shown once; reloading the page doesn't show it again
Roles	Matches the Terraform role list exactly
Approvals	Requester can't approve; a second Owner can; expired after 1 hour
Policy	Values match ZITADEL settings; drift banner shows when CI reports drift
Audit	Every action above appears within 1 minute with actor, before and after

Delivery plan

Phase	Weeks	Deliverables	Done when
1 Foundation	1–2	PostgreSQL (CloudNativePG), Kubernetes Secrets, ZITADEL Helm install, DNS and TLS, branding	Admin signs in at auth.stardatadb.com with MFA
2 Configuration as code	2–3	Terraform: project, roles, apps, machine users, policies, SMTP	`terraform plan` shows no drift in dev and prod
3 Claims	3–4	stardata-claims service, Actions v2 targets and executions	Tokens carry the component claims; Doris, Kafka, Grafana and Argo CD accept them
4 Console backend	4–6	BFF sign-in, `/api/me`, access endpoints, console DB, approvals	API tests pass for all rules in Step 5
5 Console screens	5–7	12 screens from the canvas	Playwright tests pass; design review signed off
6 Audit and monitoring	6–7	audit-forwarder, Doris table and load, dashboard, alerts	Every action appears in Audit within 1 minute
7 Hardening and go-live	8	Security checklist, load test, backup and restore drill, runbook	Checklist complete; restore drill under 1 hour

Team: 1 platform engineer (phases 1–3, 6), 1 backend engineer (Go, phases 3–4), 1 frontend engineer (phase 5), part-time security reviewer.

Open decisions

Decision	Options	Suggestion
ZITADEL version to pin	latest v4.x at start, or wait for the next minor	Pin the latest v4 patch and test Authorization v2 on it

Decision	Options	Suggestion
Configuration tool	Terraform only, or Terraform plus API bootstrap job	Terraform for everything it supports; small job only for Actions v2 if the provider lacks it
Self-registration	Open with domain allow-list, or invite only	Allow-list for company domains, invite for contractors
Approval window	1 hour or 24 hours	1 hour, with a reminder at 30 min
Audit retention	10 days	Decided: keep 10 days in Doris (daily partitions, older ones dropped automatically)
External company SSO later	Entra ID or Okta as an identity provider in ZITADEL	Out of scope now; the design allows it
Kafka auth for services	SASL/OAUTHBEARER with ZITADEL tokens, or SCRAM	OAUTHBEARER for people, SCRAM or mTLS for services